

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

Отчет о программном проекте на тему:
Разработка программного обеспечения для распределенных вычислений

Выполнил студент:

группы БКНАД222, 3 курса

Запорожченко Дмитрий Константинович

Принял руководитель проекта:

Овчинников Сергей Андреевич

Ведущий инженер

Содержание

Аннотация	3
Ключевые слова	3
1 Введение	4
2 Постановка задачи	5
3 Обзор литературы	6
4 Анализ существующих решений	7
5 Выбор инструментов разработки	10
6 Архитектура системы	11
7 Протокол распределённых вычислений	14
8 Блокчейн-реестр	18
9 Исполнитель задач	20
10 Графический интерфейс	21
11 Тестирование	22
12 Заключение	24
13 Использование инструментов искусственного интеллекта	24
Список литературы	27

Аннотация

В данной работе рассматривается платформа для организации распределённых добровольных вычислений, в которой участники сети предоставляют свои вычислительные ресурсы для решения чужих задач и получают за это вознаграждение. В отличие от классических систем подобного рода, опирающихся на центральный сервер, предлагаемая система построена на одноранговой сети равноправных узлов: любой участник может как публиковать собственные задачи, так и выполнять задачи других. Поскольку результатам анонимных участников нельзя доверять заранее, корректность вычислений устанавливается коллективно — через сопоставление результатов нескольких независимых исполнителей и учёт их репутации, а все операции с вознаграждениями фиксируются в общем проверяемом реестре. Работа демонстрирует жизнеспособность децентрализованного подхода к добровольным вычислениям и может служить основой для дальнейших исследований и учебной платформой для изучения принципов построения подобных систем.

Ключевые слова

Распределённые вычисления, одноранговая сеть, BOINC, машинное обучение, блокчейн, репутационный консенсус, византийская устойчивость, изолированное исполнение кода, токеновая экономика.

1 Введение

Одной из актуальных задач является разработка и усовершенствование различных вычислительных систем, способных облегчить решение сложных задач и обеспечить эффективное использование вычислительных ресурсов. Распределённые вычислительные сети представляют собой один из наиболее ярких примеров таких систем: они объединяют большое количество географически разнесённых компьютеров в единый вычислительный комплекс, способный решать задачи, превосходящие по сложности возможности любого отдельно взятого устройства.

Одним из наиболее известных представителей данного класса систем является проект BOINC (Berkeley Open Infrastructure for Network Computing) [19], который применяется для решения научных задач самой разной природы - от расшифровки генома и поиска новых лекарств до моделирования астрофизических процессов. Идея, на которой построен BOINC, заключается в том, что любой владелец персонального компьютера может отдать часть своих простаивающих вычислительных ресурсов в пользу научного проекта и таким образом сделать вклад в общее дело. За годы существования платформы было реализовано множество успешных проектов, использующих её инфраструктуру [1].

Однако, несмотря на популярность подобных систем, существующие реализации обладают рядом ограничений. Во-первых, классическая архитектура BOINC основана на централизованной модели: каждый научный проект разворачивает собственный сервер, который занимается распределением задач, верификацией результатов и учётом вкладов волонтёров [3]. Такая модель накладывает высокие требования на администратора проекта, требует значительных операционных издержек и создаёт единую точку отказа. Во-вторых, поскольку каждый проект изолирован, отсутствует общий протокол вознаграждения, который позволил бы участнику переносить накопленный «капитал доверия» между проектами или направлять заработанные ресурсы на финансирование других задач.

Предлагаемая курсовая работа направлена на разработку альтернативной платформы, которая устраняет указанные недостатки. Разработанная система построена на полностью децентрализованной одноранговой архитектуре, в которой состояние сети реплицируется на всех узлах и согласуется византийски-устойчивым консенсусом, использует репутационно-взвешенную верификацию результатов и ведёт учёт вознаграждений во встроенном блокчейн-реестре. Пользовательские задачи — в первую очередь задачи машинного обучения (распределённое обучение, инференс) и математические вычисления — задаются в виде Python-кода и выполняются в изолированном интерпретаторе, что обеспечивает безопасность хост-системы без зависимости от внешней инфраструктуры. Реализация выполнена на языке программирования Rust [10].

Практическая ценность подобного подхода определяется двумя его свойствами. Во-первых, единая кодовая база масштабируется в широком диапазоне сценариев развёртывания: одна и та

же сеть может работать локально внутри организации — как частный вычислительный кластер из доверенных машин, — и глобально, по модели BOINC, объединяя добровольцев со всего мира. Во-вторых, встроенная токеновая экономика, в которой выполнение задачи сжигает валюту у её автора и начисляет исполнителю, а стоимость определяется фактически затраченной вычислительной работой, естественным образом отражает относительную ресурсоёмкость задач: более тяжёлые вычисления обходятся дороже, что даёт прозрачный механизм приоритизации и справедливого распределения ресурсов сети. Целью настоящей работы является создание прототипа подобной системы и проверка работоспособности предложенных решений на наборе модульных тестов.

2 Постановка задачи

Необходимо разработать программное обеспечение для организации распределённых добровольных вычислений, способное распределять пользовательские задачи между участниками одноранговой сети и обеспечивать корректность их выполнения. Система должна работать без единого центрального сервера: каждый узел сети должен иметь возможность выступать в роли координатора (публикующего задачи) или исполнителя (выполняющего задачи) в зависимости от конфигурации. Помимо ядра протокола, необходимо разработать графический клиент, позволяющий пользователю подключаться к сети, создавать проекты, публиковать задачи и мониторить состояние сети.

Функциональные требования к разрабатываемой системе формулируются следующим образом. Прежде всего, координатор должен иметь возможность регистрировать проекты и пополнять их баланс из собственных средств, после чего публиковать в рамках проекта пользовательские задачи с указанием размера вознаграждения и текста скрипта. Каждая задача должна назначаться по очереди нескольким независимым исполнителям, которые независимо вычисляют результат и отправляют координатору хэш полученного ответа. Координатор принимает результат как корректный, если суммарная репутация исполнителей, отправивших одинаковый хэш, достигает заранее заданного порога кворума. Все финансовые операции — как начальное зачисление токенов, так и выплаты вознаграждения и штрафы за недобросовестное поведение — должны фиксироваться в неизменяемом блокчейн-реестре, обеспечивающем возможность последующего аудита. Исполнитель, которому назначена задача, должен подтвердить её выполнение в течение ограниченного интервала времени (аренды); при нарушении срока он должен подвергаться штрафу, а задача — возвращаться в очередь для повторного назначения. Помимо протокольной части, система должна предоставлять кроссплатформенный графический интерфейс, позволяющий выполнять перечисленные действия в удобной для пользователя форме.

Нефункциональные требования также носят существенный характер. Реализация должна быть кроссплатформенной и собираться без модификации исходного кода на операционных системах семейства macOS, Linux и Windows. Сетевое взаимодействие между узлами должно осуществляться

поверх TCP, а сообщения — сериализоваться в текстовом формате JSON, что упрощает отладку и анализ трафика. Хранилище блокчейн-реестра должно поддерживать криптографическую проверку целостности, позволяющую обнаружить любое постфактумное вмешательство в историю транзакций. Наконец, выполнение пользовательских задач должно происходить в условиях, исключающих возможность негативного воздействия на устройство исполнителя, что подразумевает использование некоторой формы изолированного окружения.

3 Обзор литературы

Изучение существующих решений началось с работы [1], в которой описана архитектура платформы BOINC и принципы её работы. Из данной публикации было выявлено, что каждый проект BOINC управляет собственным сервером, который рассылает задачи волонтёрам с учётом характеристик их оборудования и предпочтений. Полученная информация позволила сформулировать идею о хранении метаданных об участниках сети и о фильтрации задач по их аппаратным требованиям. Развитие данной идеи приведено в более поздней статье того же автора [2], где обсуждается глобальное планирование вычислительных задач и предлагаются модели справедливого распределения нагрузки между конкурирующими проектами.

Существенную роль в формировании архитектурных требований сыграла статья [16], посвящённая опыту разработки распределённой сети The Condor. Из этой работы были позаимствованы базовые принципы управления ресурсами участников: право исполнителя в любой момент отказаться от задачи, необходимость многоуровневой изоляции пользовательского кода и обязательность механизмов обнаружения сбойных узлов. Авторы The Condor подробно описывают проблемы, возникающие при выполнении произвольного пользовательского кода на устройстве исполнителя, и предлагают использовать многоуровневую песочницу для ограничения доступа задачи к файловой системе и сети.

Вопросы экономического планирования в распределённых средах рассматриваются в работе [20], где предлагается модель планирования потока заданий и справедливого разделения ресурсов в распределённых вычислительных средах, позволяющая учитывать требования пользователей виртуальной организации к эффективности и качеству выполнения своих заданий. На основе данной работы в настоящем проекте была реализована система квот и токенового вознаграждения, где каждая задача имеет указанную постановщиком стоимость, а исполнители получают вознаграждение пропорционально вкладу.

Для проектирования механизма репутации использовалась работа [9], в которой описан алгоритм EigenTrust для управления репутацией в одноранговых сетях. В отличие от полной реализации алгоритма EigenTrust, требующего итеративного решения задачи о собственном векторе матрицы доверия, в настоящей работе применяется упрощённая схема: репутация участника представляется

неотрицательным целым числом, увеличивающимся при подтверждении результата и уменьшающимся при отклонении или истечении аренды. Это решение позволило сохранить ключевые свойства репутационного механизма при существенной экономии вычислительных ресурсов.

Теоретическим основанием для разработки механизма консенсуса послужили классические работы по согласованию в распределённых системах. Прежде всего, это работа [11], в которой сформулирована задача византийских генералов и доказана невозможность достижения согласия более чем с одной третью предателей в синхронной сети. Более практически ориентированный протокол согласования описан в работе [4], представляющей алгоритм Practical Byzantine Fault Tolerance. Хотя полное применение PBFT в рамках курсовой работы не предполагалось, идеи кворумного голосования и весовой суммы голосов нашли отражение в реализованном механизме консенсуса.

Наконец, для проектирования блокчейн-реестра была изучена основополагающая работа [14], в которой описана идея неизменяемой цепочки блоков, связанных через криптографические хэши, а также работа Меркла о криптографических деревьях [13]. В качестве базовой хэш-функции для построения цепочки выбрана SHA-256, описанная в стандарте [15]. Вопрос безопасного выполнения произвольного кода традиционно решается средствами контейнеризации, описанными в работе [12] и документации Docker [5]; в настоящей работе применён более лёгкий подход — исполнение кода в ограниченном интерпретаторе без внешних зависимостей, изложенный в разделе об исполнителе задач.

4 Анализ существующих решений

Прежде чем приступить к проектированию собственной системы, были изучены наиболее значимые существующие платформы для организации распределённых добровольных вычислений. Целью анализа являлось выявление ключевых ограничений имеющихся решений и обоснование архитектурных решений, принятых в настоящей работе.

BOINC. Платформа BOINC [19, 1] является де-факто стандартом в области добровольных вычислений и насчитывает десятки активных научных проектов. Тем не менее её архитектура основана на строго централизованной модели: каждый проект разворачивает и администрирует собственный сервер, который является единственной точкой распределения задач и верификации результатов. Это создаёт две принципиальные проблемы. Во-первых, при недоступности сервера весь проект оказывается неработоспособным; во-вторых, система кредитов BOINC является сугубо внутривнутрипроектной — накопленная репутация не переносится между проектами, и единого финансового реестра не существует. Кроме того, для публикации нового вычислительного проекта требуется развернуть собственную серверную инфраструктуру, что значительно повышает порог входа.

Folding@home. Проект Folding@home [7] специализируется на моделировании сворачивания белков и добился значительных научных результатов благодаря многомиллионному сообществу

волонтеров. Однако архитектура проекта полностью централизована, задачи носят прикладной характер (только молекулярно-динамическое моделирование), система не поддерживает публикацию произвольных пользовательских задач, а финансовые стимулы для волонтеров полностью отсутствуют.

Golem Network. Golem [17] является наиболее близким по идеологии к разработанной системе: это децентрализованная P2P-сеть, позволяющая арендовать вычислительные ресурсы других участников за вознаграждение. Тем не менее Golem обладает рядом характеристик, снижающих его практичность в образовательном и исследовательском контексте. Во-первых, платёжная система построена поверх публичного блокчейна Ethereum, что предполагает наличие у каждого участника криптовалютного кошелька, оплату комиссий за газ и подверженность волатильности курса токена GLM. Во-вторых, Golem не реализует встроенного механизма консенсуса для верификации результатов: корректность вычислений остаётся на ответственности постановщика задачи. В-третьих, инфраструктура проекта значительно сложнее в развёртывании.

GridCoin. GridCoin [8] представляет собой попытку ввести криптовалютное вознаграждение за участие в проектах BOINC посредством механизма Proof-of-Research. Однако данная система не устраняет централизованный характер самой BOINC: участник по-прежнему должен подключаться к серверам отдельных проектов, а GridCoin лишь добавляет финансовый слой поверх существующей инфраструктуры, унаследовав все её ограничения.

Сводное сравнение перечисленных платформ с разработанной системой приведено в таблице 4.1.

Таблица 4.1: Сравнение платформ для организации распределённых вычислений

Критерий	BOINC	Folding@home	Golem	Наша система
Архитектура	Централиз.	Централиз.	P2P	P2P
Единая точка отказа	Есть	Есть	Нет	Нет
Универсальность задач	Ограничена	Нет	Да	Да
Изоляция кода	Частичная	Нет	Docker/VM	Изолир. интерпретатор
Верификация результатов	Дублирование	Сервер	Нет	Репут. консенсус
Встроенная экономика	Кредиты	Нет	Внешний блокчейн	Встроенный блокчейн
Устойчивость к Byzantine	Нет	Нет	Нет	Да
Крипто-кошелёк	Не нужен	Не нужен	Обязателен	Не нужен

Анализ таблицы позволяет выделить ключевые преимущества разработанной системы над рассмотренными аналогами. По сравнению с BOINC и Folding@home система полностью избавлена от централизованного сервера и единой точки отказа: при выходе любого узла из строя сеть продолжает функционировать, поскольку любой оставшийся узел может принять роль координатора. Встроенный блокчейн-реестр обеспечивает сквозную прозрачность всех финансовых операций без зависимости от внешних платёжных систем, а репутационно-взвешенный консенсус — единственный из рассмотренных механизмов — обеспечивает автоматическую верификацию результатов с устойчивостью к недобросовестным участникам. По сравнению с Golem система существенно ниже

по барьеру входа: не требует криптовалютного кошелька, не зависит от состояния публичной сети Ethereum и не предполагает никаких транзакционных комиссий. Наконец, исполнение пользовательского кода в изолированном интерпретаторе с белым списком импортов и жёстким ограничением числа операций обеспечивает безопасность хост-системы без зависимости от внешней инфраструктуры контейнеризации, что упрощает развёртывание как внутри организации, так и на машинах добровольцев.

5 Выбор инструментов разработки

К выбору инструментов разработки было решено подойти ответственно, поскольку он во многом определяет надёжность и производительность итогового продукта. В качестве основного языка программирования был выбран Rust [10] — системный компилируемый язык, ориентированный на разработку высокопроизводительных систем с гарантиями безопасности памяти. Этот выбор обосновывается несколькими соображениями. Прежде всего, гарантии безопасности памяти, обеспечиваемые системой владения и проверкой заимствований на этапе компиляции, позволяют избежать целого класса ошибок, характерных для языков с ручным управлением памятью, без накладных расходов на сборщик мусора, типичных для языков высокого уровня. Помимо этого, экосистема Rust содержит хорошо проработанные библиотеки для асинхронного программирования, криптографии, работы с сетью и графическим интерфейсом, что позволило реализовать все необходимые компоненты системы без привлечения сторонних программ на других языках.

Для организации конкурентного выполнения сетевых операций был использован асинхронный рантайм Tokio [18] — наиболее распространённое в экосистеме Rust решение для построения сетевых сервисов. Tokio предоставляет эффективный планировщик задач, способный обслуживать большое количество одновременных соединений на ограниченном числе системных потоков, а также богатый набор примитивов синхронизации, таких как многопроизводительные каналы MPSC, широковещательные каналы и асинхронные мьютексы. Использование Tokio позволило построить сетевой слой системы по событийной модели, в которой каждое входящее соединение обслуживается независимой задачей, а обмен сообщениями между задачами и графическим интерфейсом происходит через каналы.

Для разработки графического интерфейса была выбрана библиотека egui [6], реализующая немедленный (immediate-mode) подход к построению GUI: интерфейс не описывается как иерархия объектов-виджетов, а перерисовывается полностью на каждом кадре исходя из текущего состояния программы. Такой подход избавляет разработчика от необходимости синхронизации состояния модели и виджетов и существенно упрощает реализацию динамических интерфейсов с часто изменяющимися данными. Поскольку библиотека написана на чистом Rust и не зависит от системных GUI-фреймворков, оконная оболочка eframe позволяет собирать одно и то же приложение под macOS, Linux и Windows без модификации исходного кода.

В качестве хэш-функции для построения блокчейн-реестра был выбран алгоритм SHA-256, описанный в стандарте [15] и реализованный в крейте sha2 из набора RustCrypto. Выбор именно SHA-256, а не более современных функций семейства SHA-3 или BLAKE3, обусловлен повсеместной распространённостью данного алгоритма, его хорошо изученными криптографическими свойствами и широкой поддержкой в стандартных библиотеках различных языков, что облегчает возможную интероперабельность с другими реализациями.

Для исполнения пользовательских Python-задач применяется интерпретатор CPython, встроенный непосредственно в приложение через привязки PyO3. Такое решение, в отличие от первоначально рассматривавшейся контейнеризации Docker [12, 5], не требует наличия в системе стороннего демона и работает единообразно на всех целевых платформах, что существенно снижает порог входа: пользователю достаточно запустить один исполняемый файл. Безопасность при этом обеспечивается не средствами операционной системы, а на уровне самого интерпретатора — через изолированный исполнитель кода из библиотеки `smolagents`, который ограничивает множество доступных импортов, запрещает доступ к небезопасным подмодулям и накладывает жёсткий предел на число выполняемых операций, прерывая в том числе бесконечные циклы. Чтобы исключить зависимость от системного Python, приложение поставляется с самодостаточным дистрибутивом интерпретатора, а устанавливаемые пакеты размещаются в изолированном виртуальном окружении.

6 Архитектура системы

Архитектура разработанной системы основана на одноранговой (peer-to-peer) модели, в которой отсутствует единый центральный сервер, а каждый узел сети может одновременно выступать и постановщиком задач, и их исполнителем. Принципиальной особенностью является то, что состояние сети — список участников, проекты, задачи, репутация и блокчейн-реестр — не хранится на выделенном сервере, а реплицируется на всех узлах: каждый узел поддерживает собственную полную копию и применяет к ней изменения в едином согласованном порядке (подробнее см. раздел 7). Координатор в этой модели — лишь точка входа, принимающая TCP-соединения и помогающая новым участникам синхронизироваться, а не привилегированный владелец данных. В рамках одного процесса допускается одновременная работа нескольких логических узлов, что удобно для локального тестирования.

Приложение собирается в виде единого бинарного файла с двумя точками запуска: десктопным графическим клиентом и headless-узлом для командной строки. Второй вариант использует тот же сетевой и вычислительный движок, но не требует графической среды, что позволяет разворачивать узлы на серверах и в составе автоматизированных сценариев тестирования консенсуса.

Внутренняя организация приложения построена на разделении ответственности между двумя параллельными контекстами исполнения. Главный поток операционной системы занят циклом событий графического интерфейса: библиотека `eframe` требует, чтобы все вызовы `egui` производились из основного потока, поскольку многие графические API ведут себя корректно только в таком режиме. Параллельно с главным потоком работает пул потоков асинхронного рантайма `Tokio`, на котором запущен сетевой актор — центральный компонент, реализующий протокол взаимодействия и хранящий состояние сети. Между графическим интерфейсом и сетевым актором установлена пара каналов MPSC: командный канал, по которому GUI отправляет в сетевой слой запросы пользователя

(создать проект, опубликовать задачу, подключиться к узлу), и канал событий, по которому сетевой слой сообщает в GUI об изменениях состояния (поступление новой задачи, изменение баланса, обновление списка участников). Использование каналов позволяет полностью развязать графический интерфейс и сетевую логику: GUI остаётся отзывчивым даже при длительных сетевых операциях, а сетевой слой может работать независимо от наличия активного интерфейса.

Сетевой актор внутри себя мультиплексирует два режима работы. В режиме координатора актор открывает прослушивающий TCP-сокеты, ожидает подключений и для каждого нового соединения запускает отдельную асинхронную задачу, обрабатывающую входящие сообщения. В режиме исполнителя актор поддерживает одно исходящее соединение с координатором, периодически запрашивает новые задачи и обрабатывает входящие сообщения от координатора. Для согласованного изменения общего состояния между обработчиками соединений используется широковещательный канал, по которому рассылаются обновления состояния всем заинтересованным сторонам. Сериализация сообщений выполняется в формате JSON, а разделение потока данных на отдельные сообщения — с использованием кодека LinesCodec из библиотеки tokio-util, что обеспечивает простоту отладки и совместимость с инструментами анализа сетевого трафика.

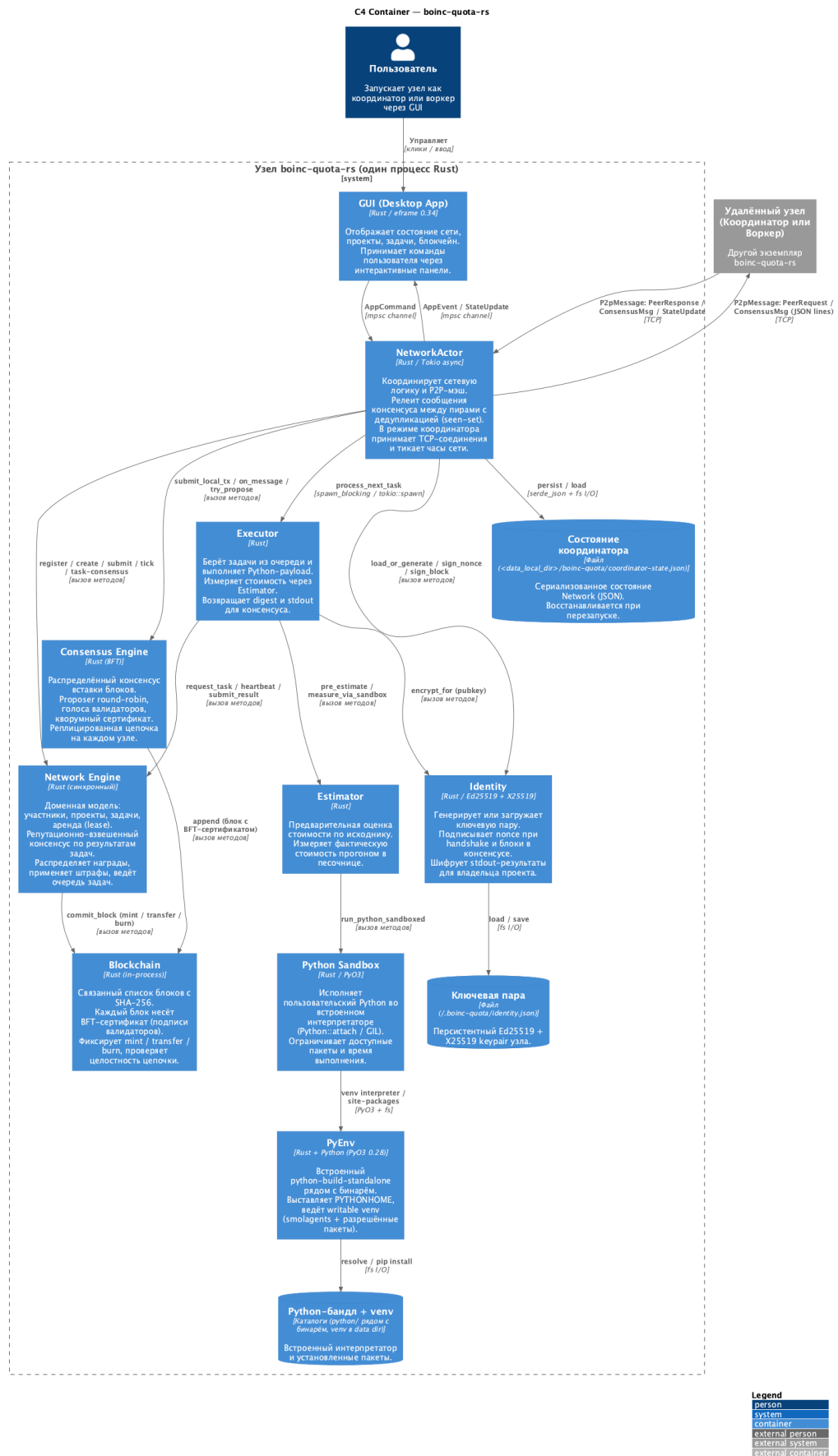


Рис. 6.1: C4-диаграмма контейнеров: компоненты узла и их взаимодействие

Поскольку подобная одноранговая архитектура в существенной мере отличается от классиче-

ской клиент-серверной модели, применяемой в большинстве реализаций распределённых сетей, она потребовала проработки ряда нетривиальных вопросов, связанных с согласованием состояния между узлами и обнаружением выхода из строя отдельных участников. Эти вопросы рассматриваются в следующем разделе.

7 Протокол распределённых вычислений

Протокол системы решает две взаимосвязанные задачи: согласование общего состояния сети между всеми узлами и верификацию результатов вычислений. Им соответствуют два механизма консенсуса, действующих на разных уровнях.

Согласование состояния. Все изменения общего состояния выражаются последовательностью операций (регистрация участника, создание проекта, публикация задачи, начисление вознаграждения и т. п.). Каждый узел применяет эту последовательность к собственной полной копии состояния в одном и том же порядке, благодаря чему все узлы детерминированно сходятся к байт-идентичному результату: идентификаторы проектов и задач выделяются последовательными счётчиками, которые продвигаются синхронно именно потому, что порядок операций одинаков на всех узлах. Согласованный порядок операций устанавливается византийски-устойчивым консенсусом вставки блоков. Для каждой следующей высоты цепочки назначается узел-предложитель (по круговому принципу из набора валидаторов), который формирует блок-кандидат, подписывает его и рассылает остальным; валидаторы голосуют за хэш блока, и как только голоса достигают кворума, блок запечатывается набором подписей (сертификатом) и добавляется в цепочку у всех узлов. Сообщения консенсуса распространяются по сети методом лавинной рассылки с дедупликацией уже виденных сообщений, а накопленная цепочка сохраняется между переподключениями узла. Идейно описанный подход опирается на классические работы по византийской устойчивости [11, 4].

Верификация результатов. Корректность самих вычислений устанавливается отдельным репутационно-взвешенным консенсусом, реализованным в модуле `network` и описанным далее. Жизненный цикл каждой задачи описывается конечным автоматом с четырьмя основными состояниями. После публикации задача оказывается в состоянии «ожидает назначения» (Pending) и помещается в очередь координатора. В этот момент сумма вознаграждения, указанная постановщиком, резервируется на проектном счёте: квота `quota_available` проекта уменьшается, а `quota_locked` соответственно увеличивается, что фиксируется в блокчейн-реестре как транзакция `Transfer` с проектного счёта на временный резерв задачи. Когда свободный исполнитель запрашивает задачу, координатор выбирает её из очереди по приоритету: проекты с большим объёмом неосвоенной квоты получают преимущество, а внутри проекта — задачи с большим идентификатором, что обеспечивает приблизительную справедливость FIFO. Дополнительное условие выбора — исполнитель ранее не присылал отчёт по этой же задаче, чтобы избежать ситуации, в которой одна и та же сторона голосует

за результат несколько раз. После выбора задачи она переходит в состояние `Assigned`, и исполнителю выдаётся аренда сроком `lease_timeout_ticks` тиков. Тик является дискретной единицей логического времени симуляции: счётчик тиков продвигается явными вызовами `network.tick(n)` и не привязан к астрономическому времени. Такой подход позволяет тестировать временные инварианты — истечение аренды, частоту heartbeat-сообщений — детерминированно и без использования системных таймеров.

Когда исполнитель завершает вычисления, он отправляет координатору отчёт о выполнении, содержащий хэш результата (`digest`), идентификатор задачи и измеренные параметры потребления ресурсов. Координатор регистрирует отчёт и проверяет, не достигнут ли консенсус по одному из пришедших хэшей. Механизм консенсуса является репутационно-взвешенным: каждый отчёт вносит в общую сумму голосов вес, равный текущей репутации соответствующего исполнителя, после чего проверяется, превысила ли суммарная репутация для какого-либо хэша заданный порог кворума (по умолчанию — 2). Если порог достигнут, задача переходит в состояние `Completed`: согласившимся исполнителям начисляется вознаграждение, их репутация увеличивается, а несогласные с консенсусом исполнители получают штраф. Если же ни один хэш не набрал кворума, но количество отчётов уже превысило заданный лимит попыток, задача переходит в состояние `Rejected`, а зарезервированная сумма возвращается на проектный счёт.

Отдельного внимания заслуживает механизм аренды, обеспечивающий отказоустойчивость системы. После назначения задачи исполнителю запускается таймер: если в течение `lease_timeout_ticks` тиков от него не было получено ни одного отчёта или сообщения о продлении аренды (`heartbeat`), координатор считает аренду истёкшей. Истечение аренды интерпретируется как факт недобросовестного поведения или сбоя у исполнителя: его репутация уменьшается, с его балансовой записи списывается штраф в виде транзакции `Burn`, а сама задача возвращается в очередь `Pending` для повторного назначения. Такой механизм позволяет системе автоматически восстанавливаться после сбоев исполнителей, не требуя явного вмешательства администратора. Полный набор состояний задачи и переходов между ними приведён на рисунке 7.1.

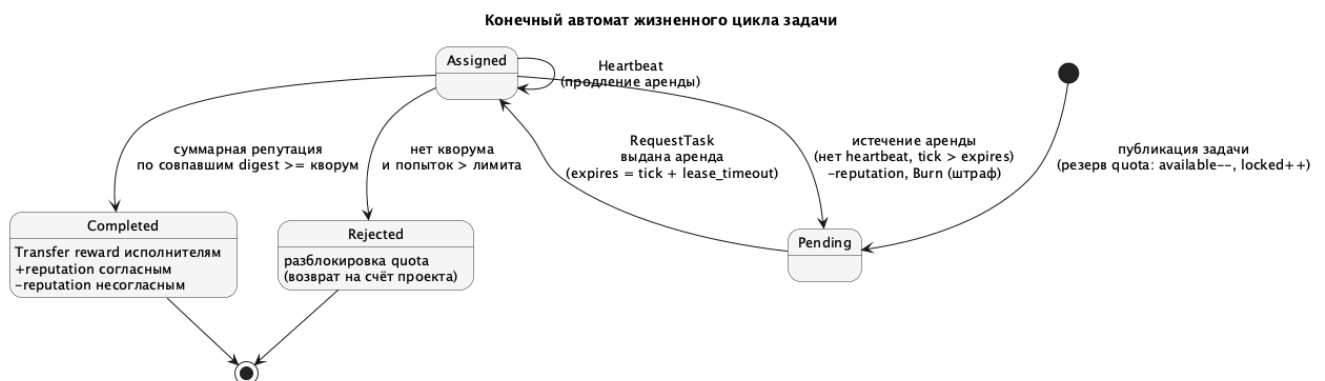


Рис. 7.1: Конечный автомат жизненного цикла задачи

Для тестирования устойчивости системы к недобросовестным участникам в исполнителе реализован режим симуляции византийского поведения. При настройке параметра надёжности менее 100% исполнитель с заданной вероятностью отправляет заведомо искажённый хэш результата, что моделирует ситуацию преднамеренного саботажа со стороны участника. Поведение исполнителя при этом остаётся детерминированным и воспроизводимым при фиксированном начальном числе псевдослучайности, что облегчает воспроизведение тестовых сценариев. Подобный режим позволяет на практике убедиться в том, что репутационный механизм консенсуса корректно отсекает влияние недобросовестных голосов и в долгосрочной перспективе приводит к снижению их репутации до нуля, после чего их голоса перестают учитываться при кворумном голосовании. Идейно описанный подход к голосованию опирается на классические работы по византийской устойчивости [11, 4].

Полная последовательность взаимодействий между участниками сети за время жизни одной задачи — от аутентификации исполнителя и публикации задачи до достижения консенсуса и начисления вознаграждения — приведена на рисунке 7.2.

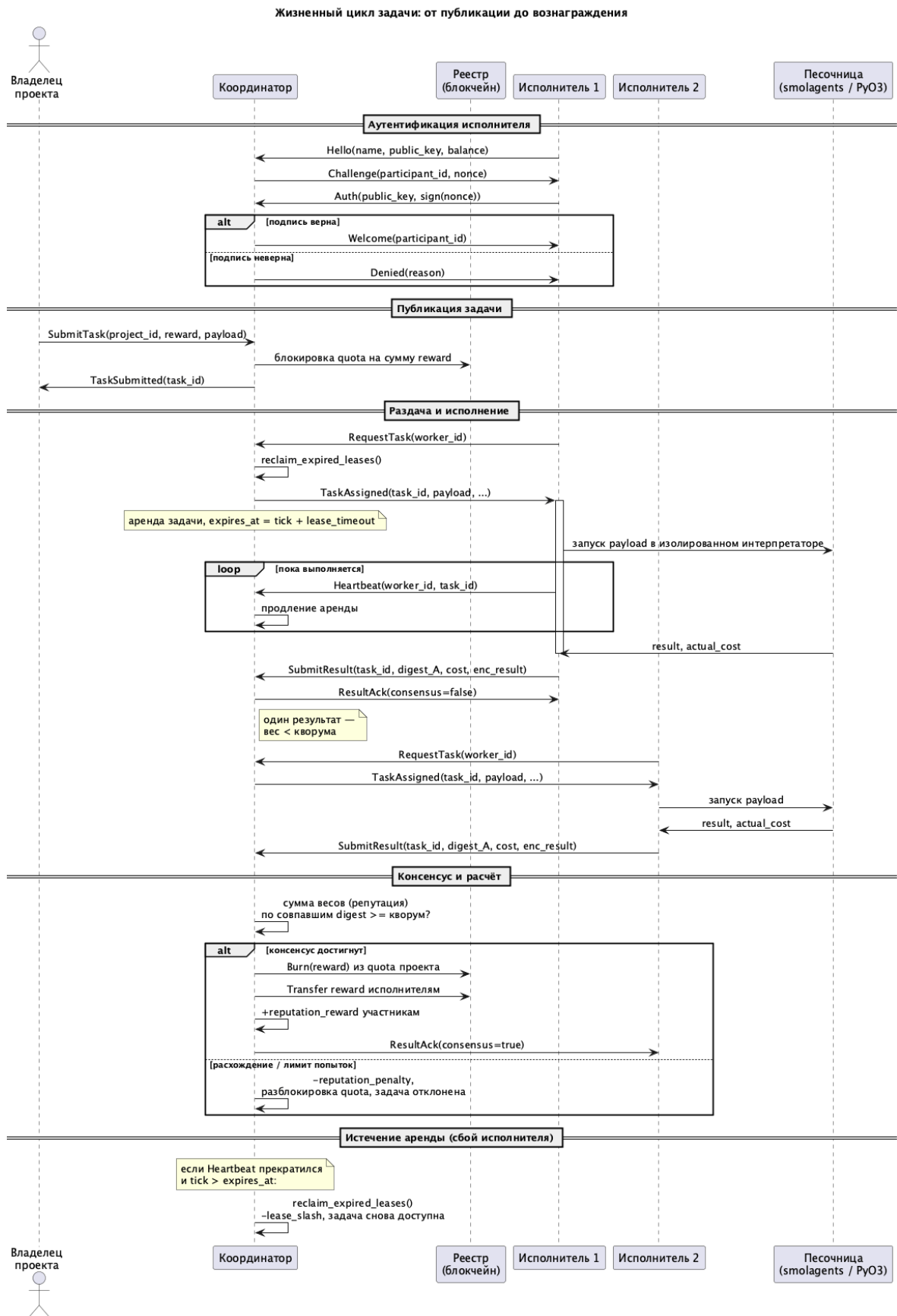


Рис. 7.2: Диаграмма последовательности жизненного цикла задачи

8 Блокчейн-реестр

Для прозрачного и проверяемого учёта всех финансовых операций в системе реализован блокчейн-реестр, представляющий собой неизменяемую цепочку блоков, связанных через криптографические хэши. Идея подобной структуры данных была предложена ещё в работах по криптографическим деревьям [13] и получила широкую известность благодаря применению в системе Bitcoin [14]. В отличие от Bitcoin, согласование цепочки между узлами достигается не доказательством работы, а византийски-устойчивым голосованием валидаторов, описанным в разделе 7: каждый блок несёт сертификат из подписей валидаторов, одобдивших его, а полная копия реестра хранится на каждом узле сети. Тем самым реестр является не вспомогательным журналом аудита, а единым источником истины об общем состоянии системы.

Каждый блок реестра содержит порядковый номер (высоту блока), полезную нагрузку в виде описания одной транзакции и хэш SHA-256 от заголовка предыдущего блока. Хэш предыдущего блока вычисляется по конкатенации его номера, сериализованного представления транзакции и его собственного предшествующего хэша, что обеспечивает рекурсивную связность всей цепочки. Любая попытка изменить содержимое исторического блока приводит к изменению его хэша, что в свою очередь инвалидирует хэш-связку со следующим блоком, и так до конца цепочки. Таким образом, для подделки одной исторической записи злоумышленнику потребовалось бы пересчитать хэши всех последующих блоков, что и используется в качестве свойства неизменяемости. Для криптографической части реестра применяется алгоритм SHA-256, описанный в стандарте [15], как наиболее распространённый и хорошо изученный. Структура хэш-связности блоков и механизм обнаружения подмены показаны на рисунке 8.1.

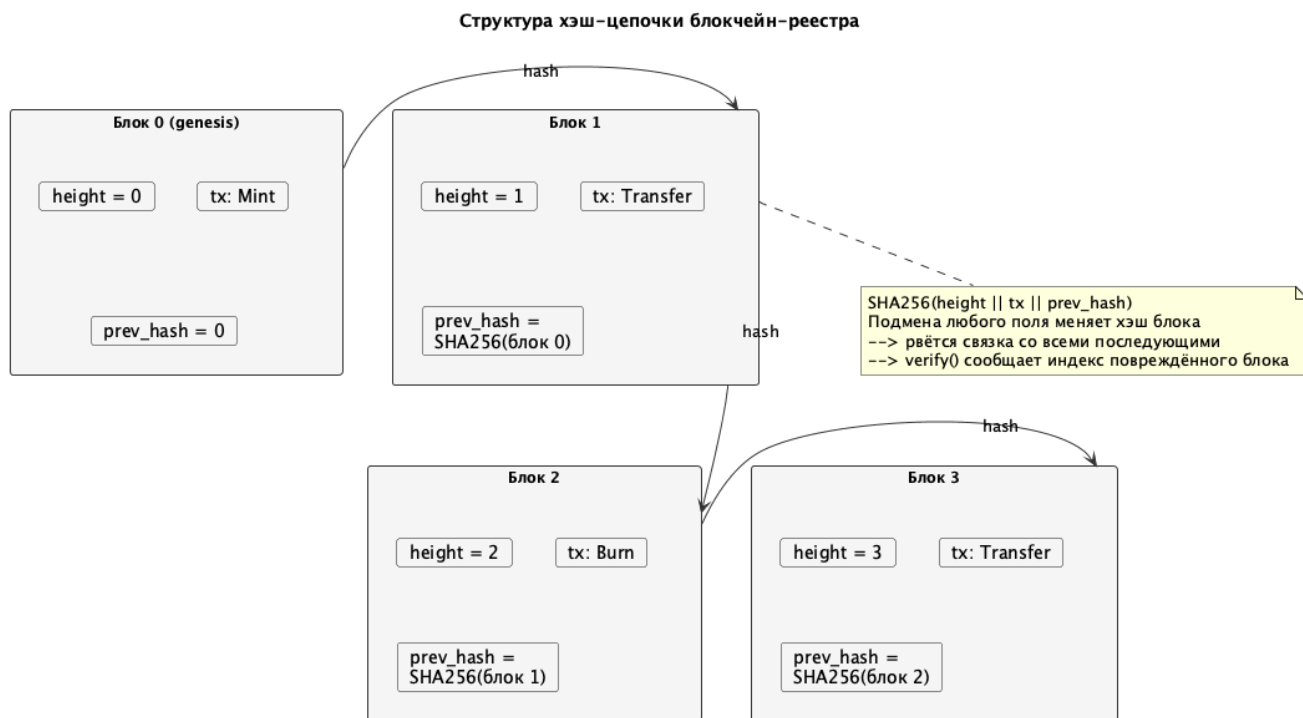


Рис. 8.1: Структура хэш-цепочки блокчейн-реестра

В реестре поддерживаются три типа транзакций. Транзакция Mint описывает начальное зачисление токенов вновь зарегистрированному участнику и применяется один раз при создании учётной записи. Транзакция Transfer описывает перевод токенов между двумя адресами и используется как для финансирования проектов постановщиками, так и для выплаты вознаграждения исполнителям после успешного завершения задачи. Транзакция Burn описывает уничтожение токенов и применяется в качестве меры наказания за истечение аренды или отправку результата, не совпадающего с консенсусом. Для разделения адресных пространств участников и проектов проектные счета имеют отдельные адреса, формируемые добавлением константного смещения 10^9 к идентификатору проекта, что исключает коллизии с адресами участников при разумных размерах сети.

Верификация целостности реестра выполняется методом, который последовательно проходит все блоки и для каждого вычисляет ожидаемый хэш на основе хранимого содержимого, сравнивая его с фактическим хэшем-связкой следующего блока. При обнаружении любого несоответствия возвращается ошибка с указанием индекса повреждённого блока, что позволяет точно локализовать место постфактумного вмешательства. Данная процедура запускается при каждом запросе баланса, что гарантирует, что итоговые суммы вычисляются по проверенной истории. Текущий баланс конкретного адреса определяется как алгебраическая сумма всех транзакций, в которых данный адрес участвовал в качестве отправителя или получателя, что соответствует модели «полного состояния», а не модели непотраченных выходов (UTXO), используемой в Bitcoin.

9 Исполнитель задач

Компонент `executor` отвечает за непосредственное выполнение задач, назначенных исполнителю. Задача представляет собой полезную нагрузку с пользовательским Python-кодом, передаваемым в кодировке `base64`; префикс полезной нагрузки (`python:` или `python-gpu:`) указывает желаемый режим вычислений. Ориентация именно на Python обусловлена целевым классом задач — машинным обучением и научными вычислениями, — для которого этот язык является де-факто стандартом и обладает богатой экосистемой библиотек.

Изолированное исполнение. Код задачи выполняется во встроенном в приложение интерпретаторе CPython через привязки PyO3, без запуска внешних процессов или контейнеров. Безопасность обеспечивается изолированным исполнителем кода из библиотеки `smolagents` (`LocalPythonExecutor`) который интерпретирует код в строго ограниченной среде. Импорт модулей разрешён только из явного белого списка; доступ к небезопасным подмодулям (например, `random._os`) блокируется даже для разрешённых модулей, что закрывает обходные пути к функциям операционной системы. На число выполняемых интерпретатором операций наложен жёсткий предел, который останавливает в том числе бесконечные циклы без принудительного завершения процесса; дополнительной страховкой служит ограничение по реальному времени выполнения. Вывод `print` перехватывается в буфер и возвращается как результат задачи, а не печатается в консоль хоста. Тем самым достигается уровень изоляции, достаточный для запуска непроверенного кода, без зависимости от средств виртуализации операционной системы. Вложенность слоёв изоляции — от хост-системы до пользовательского кода — показана на рисунке 9.1.

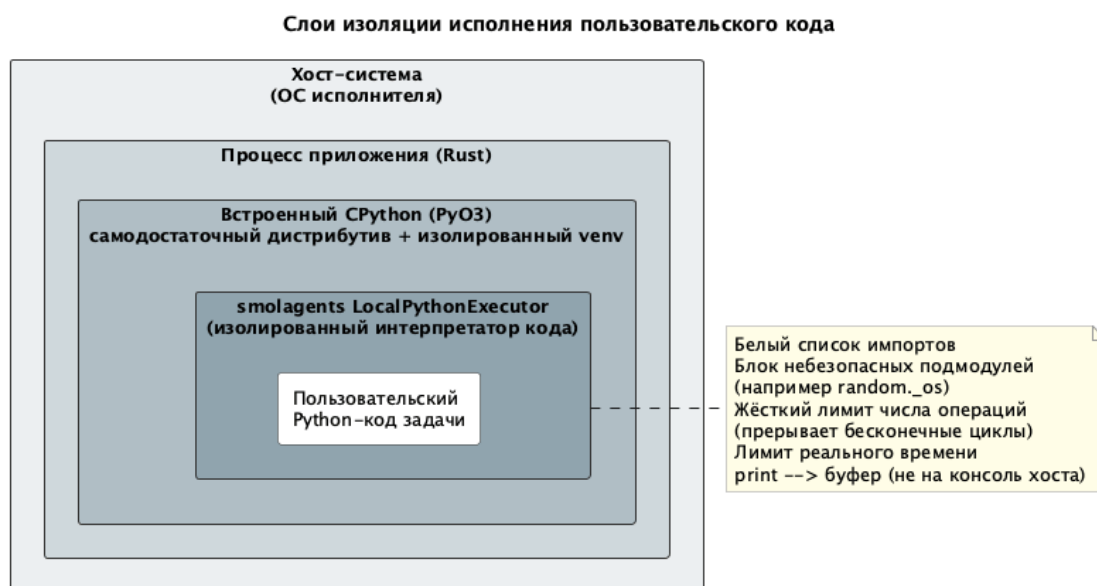


Рис. 9.1: Слои изоляции исполнения пользовательского кода

Окружение Python. Чтобы исполнение не зависело от наличия и версии системного Python, приложение поставляется с самодостаточным дистрибутивом интерпретатора, размещённым рядом

с исполняемым файлом; при его отсутствии используется системный Python версии не ниже 3.10. Устанавливаемые задачами пакеты (включая саму библиотеку smolagents) размещаются в отдельном виртуальном окружении в пользовательском каталоге данных, что изолирует их от системных пакетов и делает развёртывание воспроизводимым.

Оценка стоимости. Стоимость задачи вычисляет компонент estimator. До запуска он формирует предварительную оценку по тексту исходного кода, а после выполнения уточняет её по фактической вычислительной работе — счётчику операций, который ведёт интерпретатор в ходе исполнения. Эта величина детерминированно зависит от пути исполнения и потому одинакова на разных машинах, что важно для согласованности экономики. Полученная стоимость определяет, сколько валюты будет сожжено у автора задачи и начислено исполнителю; тем самым более ресурсоёмкие задачи естественным образом обходятся дороже, а сама экономика служит прозрачным индикатором относительной тяжести задач в сети.

10 Графический интерфейс

Графический интерфейс приложения реализован на библиотеке egui [6] в немедленном (immediate-mode) режиме, при котором всё состояние интерфейса полностью перерисовывается на каждом кадре исходя из текущей модели данных. Оконная оболочка eframe предоставляет нативное окно операционной системы размером 960 на 640 пикселей и реализует цикл событий, в котором происходит сначала обработка входящих событий ввода, а затем перерисовка интерфейса. Подобный подход к построению GUI существенно упрощает синхронизацию состояния: при изменении модели нет необходимости явно обновлять виджеты, поскольку на следующем кадре они будут построены заново исходя из новых данных. Это особенно полезно для приложений с часто изменяющимся состоянием, к которым относится монитор распределённой сети.

Интерфейс приложения разделён на пять вкладок, каждая из которых соответствует одной из основных задач пользователя. Вкладка Connect отвечает за установление соединения с сетью: в ней пользователь выбирает режим работы узла (координатор или исполнитель) и указывает адрес для прослушивания или подключения, после чего сетевой актор инициирует соответствующую операцию через командный канал. Вкладка Projects отображает список проектов сети с указанием названия, общего баланса и доступной квоты; координатор может создавать в ней новые проекты и пополнять их балансы. Вкладка Tasks позволяет публиковать новые задачи с указанием текста скрипта, размера вознаграждения и принадлежности к проекту, а также мониторить текущий статус существующих задач с разбивкой по состояниям. Вкладка Executor содержит настройки исполнителя: максимальное количество одновременно выполняемых задач, интервал отправки heartbeat-сообщений и уровень надёжности для симуляции византийского поведения. Наконец, вкладка Network отображает текущий список участников сети с их репутацией и балансом, а

также журнал транзакций блокчейн-реестра в удобной для чтения табличной форме.

Обновление состояния интерфейса происходит через канал событий, по которому сетевой актор присылает уведомления об изменениях. Получив такое событие, главный поток приложения вызывает методы `gui`-контекста для запроса перерисовки, в результате чего обновлённые данные отображаются на следующем кадре. Поскольку перерисовка происходит только при наличии входящих событий или ввода пользователя, в спокойном состоянии приложение практически не потребляет процессорное время, что делает его пригодным для длительной работы в фоновом режиме на устройстве пользователя.

Пример работы сети из трёх узлов приведён на рисунке 10.1. Узел в роли координатора (слева) опубликовал три задачи с Python-кодом проверки числа на простоту, а два подключённых исполнителя (справа) независимо выполнили их: для каждой задачи получен результат (`prime` или `composite`), а исполнители получили вознаграждение и репутацию.

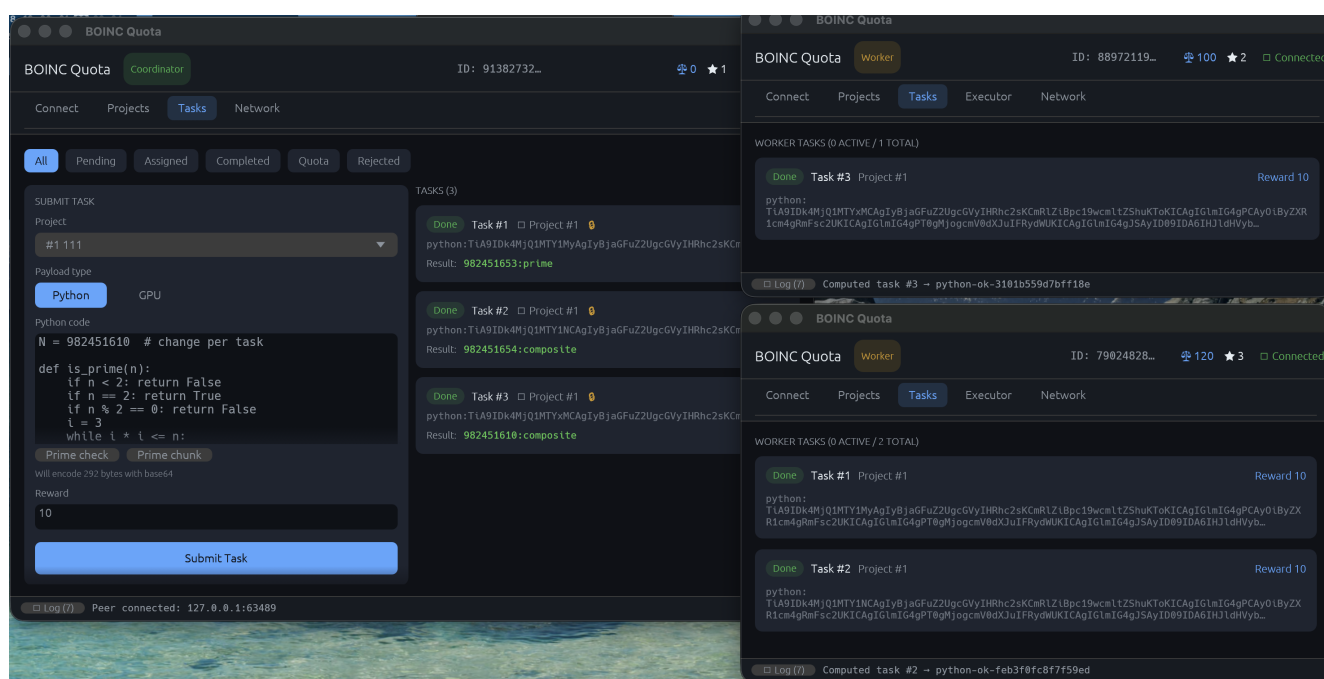


Рис. 10.1: Координатор и два исполнителя: публикация и распределённое выполнение вычислительных задач

11 Тестирование

Корректность реализации проверяется набором модульных тестов, расположенных непосредственно в исходных файлах в соответствии с принятыми в экосистеме Rust соглашениями. Общее количество тестов составляет шестьдесят, и они покрывают все основные слои системы: блокчейн-реестр, BFT-консенсус, сетевой протокол, исполнитель задач, изолированную среду исполнения, оценщик ресурсов и подсистему идентификации. Запуск всего набора производится стандартной командой `cargo test` и не требует наличия внешних зависимостей, поскольку тесты используют

режим Mock для исполнения задач и не обращаются к реальной сети.

Девять тестов посвящены проверке блокчейн-реестра. В них проверяется корректность операций Mint, Transfer и Burn, точность вычисления баланса по истории транзакций, согласованность пересечений между адресными пространствами участников и проектов, а также — что особенно важно для гарантий неизменяемости — обнаружение постфактумных подмен в исторических блоках. Один из ключевых тестов специально модифицирует поле суммы в произвольном историческом блоке и убеждается в том, что метод верификации немедленно сообщает об ошибке с указанием на номер этого блока. Подобная проверка демонстрирует, что выбранная конструкция хэш-цепочки выполняет своё назначение и обеспечивает заявленный уровень целостности данных.

Пятнадцать тестов посвящены сетевому протоколу. В них моделируется полный жизненный цикл задачи от публикации до завершения с применением мок-исполнителей, имитирующих различные сценарии поведения: совпадение результатов и достижение консенсуса, отказ части участников, истечение аренды у одного из них, расхождение результатов и отсутствие консенсуса в пределах лимита попыток, а также отклонение задачи по достижении этого лимита. Отдельная группа тестов проверяет корректность пересчёта репутации и баланса участников после каждого из перечисленных сценариев, что позволяет убедиться в согласованности протокольной части с блокчейн-реестром. Ещё девять тестов покрывают BFT-консенсус: формирование блока предложителем, голосование валидаторов, запечатывание блока при достижении кворума, сходимость цепочки на разных узлах, а также вычисление кворума по множеству активных валидаторов — в частности, что отключившиеся узлы не блокируют достижение кворума, а восстановление цепочки не возвращает их в активный набор.

Четыре теста посвящены поведению исполнителя в различных условиях: успешное выполнение задачи и получение вознаграждения, истечение аренды и связанные с этим штрафы, отсутствие доступных задач и ожидание в состоянии покоя. Десять тестов покрывают изолированную среду исполнения: они подтверждают, что запрещённые импорты (`os`, `subprocess`) и доступ к подмодулям блокируются, разрешённые модули работают, бесконечный цикл прерывается по пределу операций, а вывод и счётчик операций корректно возвращаются. Пять тестов покрывают модуль `estimator` (предварительная и фактическая оценка стоимости, детерминированность измерений) и ещё пять — подсистему идентификации (генерация и загрузка ключей, подпись и проверка, шифрование результата). Наконец, три интеграционных теста поднимают несколько узлов в одном процессе и проверяют сквозные сценарии: формирование одноранговой сети и сходимость состояния после перевода токенов, согласование проекта и задачи между координатором и исполнителями, а также полное выполнение опубликованной задачи. Прохождение всего набора тестов на момент написания работы подтверждает работоспособность ключевых компонентов системы и служит дополнительным основанием для дальнейшего развития проекта.

12 Заключение

В результате выполнения курсовой работы было реализовано программное обеспечение для организации распределённых добровольных вычислений с децентрализованной одноранговой архитектурой, написанное на языке программирования Rust. Разработанная система устраняет ряд ограничений, характерных для классических реализаций подобных платформ, основанных на централизованной клиент-серверной модели, и демонстрирует жизнеспособность альтернативного подхода. В частности, реализован протокол согласования общего состояния сети на основе византийски-устойчивого консенсуса, при котором реестр реплицируется на всех узлах, а также репутационно-взвешенная верификация результатов задач с механизмом аренды и истечением срока для автоматического восстановления после сбоев исполнителей.

Финансовая часть системы построена на основе блокчейн-реестра, обеспечивающего прозрачный и проверяемый учёт всех операций с токенами вознаграждения, что было достигнуто за счёт применения связной структуры данных на основе хэш-функции SHA-256 и встроенной процедуры верификации целостности. Безопасность исполнения пользовательских задач обеспечивается их запуском в изолированном интерпретаторе с белым списком импортов и жёстким ограничением числа операций, что исключает негативное воздействие задачи на устройство исполнителя без зависимости от внешних средств виртуализации. Взаимодействие пользователя с системой осуществляется через нативный десктопный графический интерфейс, кроссплатформенно работающий под macOS, Linux и Windows из единой кодовой базы, а для серверных развёртываний предусмотрен headless-узел.

Практическая значимость работы определяется тем, что одна и та же система масштабируется от частного кластера внутри организации до глобальной добровольной сети в духе BOINC, а встроенная экономика, привязывающая стоимость задачи к фактически затраченной вычислительной работе, даёт естественный механизм оценки относительной ресурсоёмкости задач и справедливого распределения ресурсов. Корректность реализации подтверждена набором модульных тестов, покрывающим ключевые слои системы и моделирующим типичные сценарии её работы, включая поведение в условиях частичных отказов и недобросовестного поведения участников. Полученное программное обеспечение может служить основой для дальнейших исследований в области масштабируемых децентрализованных вычислений и применяться в качестве учебной платформы. В качестве направлений дальнейшего развития можно выделить динамическую реконфигурацию набора валидаторов, расширение набора поддерживаемых вычислительных бэкендов и выделение распределённого обучения моделей в отдельный класс задач с агрегацией промежуточных результатов.

13 Использование инструментов искусственного интеллекта

В ходе разработки системы применялись языковые модели семейства Claude производства Anthropic — Claude Opus и Claude Sonnet. Инструменты ИИ использовались точно для решения

задач, не связанных с ключевой архитектурой системы: генерации кода графического интерфейса и написания модульных тестов.

Графический интерфейс. Компонент GUI построен на библиотеке `egui` с применением `immediate-mode` подхода, где каждый кадр интерфейс полностью перерисовывается. Поскольку этот компонент носит вспомогательный характер и не затрагивает логику протокола или консенсуса, его реализация была поручена языковой модели. Модель получала описание требуемых вкладок, элементов управления и связей с состоянием приложения, после чего генерировала соответствующий Rust-код.

В частности, языковой модели был передан перечень из пяти вкладок (`Connect`, `Projects`, `Tasks`, `Executor`, `Network`) с указанием отображаемых полей и доступных пользователю действий, а также описание модели данных приложения и канала событий, по которому сетевой актор уведомляет интерфейс об изменениях. На основе этого описания модель генерировала код построения виджетов, привязку элементов управления к командному каналу актора и логику обновления таблиц при поступлении событий. Сгенерированный код проходил проверку компилятором Rust и ручное тестирование во взаимодействии с реальной сетевой частью; обнаруженные несоответствия в привязке состояния устранялись последующими уточняющими запросами к модели. Такой подход позволил сосредоточить усилия на проектировании протокола и консенсуса, делегировав рутинную разработку интерфейсного слоя.

Модульные тесты. Языковая модель использовалась для написания юнит-тестов к реализованным модулям. На основе описания инвариантов и ожидаемого поведения компонентов модель генерировала тестовые сценарии, покрывающие как штатные пути выполнения, так и граничные случаи. Так, для блокчейн-реестра модели были переданы требования к операциям `Mint`, `Transfer` и `Burn` и гарантия неизменяемости хэш-цепочки, на основе чего она сформировала, в том числе, тест на обнаружение постфактумной подмены поля суммы в историческом блоке. Для сетевого протокола модель по описанию жизненного цикла задачи и возможных сценариев — достижение консенсуса, отказ участников, истечение аренды, расхождение результатов — сгенерировала соответствующие мок-исполнители и проверки пересчёта репутации и баланса. Все сгенерированные тесты проверялись на осмысленность и соответствие заявленным инвариантам, после чего включались в общий набор; именно эти тесты составляют значительную часть из шестидесяти описанных в разделе «Тестирование».

Организация рабочего процесса. Для обеспечения корректного контекста при работе с кодовой базой в корне репозитория был создан файл `CLAUDE.md`, содержащий описание архитектуры проекта, модульной структуры и ключевых инвариантов системы. Это позволило модели генерировать код, согласованный с принятыми архитектурными решениями, без необходимости передавать полный контекст в каждом запросе.

Дополнительно был настроен хук `Claude Code`, автоматически запускающий набор тестов

после каждого изменения файлов исходного кода. Такой подход позволил обнаруживать регрессии немедленно и сохранять работоспособность кодовой базы в процессе итеративной генерации кода.

Список литературы

- [1] D. P. Anderson. “BOINC: A System for Public-Resource Computing and Storage”. В: *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing*. Pittsburgh, PA, USA, 2004, с. 4—10.
- [2] D. P. Anderson. “Globally Scheduling Volunteer Computing”. В: *Future Internet* 13.9 (2021), с. 229.
- [3] BOINC Project. *BOINC overview*. URL: <https://github.com/BOINC/boinc/wiki/BOINC-overview> (дата обр. 22.05.2026).
- [4] M. Castro и B. Liskov. “Practical Byzantine Fault Tolerance”. В: *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI)*. New Orleans, USA, 1999, с. 173—186.
- [5] Docker, Inc. *Docker security documentation*. URL: <https://docs.docker.com/engine/security/> (дата обр. 20.04.2025).
- [6] E. Ernerfeldt. *egui: an easy-to-use immediate-mode GUI in pure Rust*. URL: <https://github.com/emilk/egui> (дата обр. 20.04.2025).
- [7] Folding@home Consortium. *Folding@home: fighting disease with a world wide distributed super computer*. URL: <https://foldingathome.org/> (дата обр. 20.04.2025).
- [8] Gridcoin Community. *Gridcoin: rewarding BOINC distributed computation*. URL: <https://gridcoin.us/> (дата обр. 20.04.2025).
- [9] S. D. Kamvar, M. T. Schlosser и H. Garcia-Molina. “The EigenTrust Algorithm for Reputation Management in P2P Networks”. В: *Proceedings of the 12th International World Wide Web Conference*. Budapest, Hungary, 2003, с. 640—651.
- [10] S. Klabnik и C. Nichols. *The Rust Programming Language*. 2-е изд. San Francisco: No Starch Press, 2023, с. 560.
- [11] L. Lamport, R. Shostak и M. Pease. “The Byzantine Generals Problem”. В: *ACM Transactions on Programming Languages and Systems* 4.3 (1982), с. 382—401.
- [12] D. Merkel. “Docker: lightweight Linux containers for consistent development and deployment”. В: *Linux Journal* 2014.239 (2014).
- [13] R. C. Merkle. “Secrecy, Authentication, and Public Key Systems”. В: *Stanford University Ph.D. Dissertation* (1979).
- [14] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. 2008. URL: <https://bitcoin.org/bitcoin.pdf> (дата обр. 20.04.2025).
- [15] National Institute of Standards and Technology. *Secure Hash Standard (SHS)*. Tex. отч. FIPS PUB 180-4. Gaithersburg, MD: U. S. Department of Commerce, 2015.

- [16] D. Thain, T. Tannenbaum и М. Livny. “Distributed Computing in Practice: the Condor Experience”. В: *Concurrency and Computation: Practice and Experience* 17.2–4 (2005), с. 323—356.
- [17] The Golem Project. *The Golem Project: A Decentralized Supercomputer (Crowdfunding Whitepaper)*. Тех. отч. Golem Factory GmbH, 2016. URL: https://assets.website-files.com/62446d07873fde065cb62446d07873fdeb626bcb927_Golemwhitepaper.pdf (дата обр. 22.05.2026).
- [18] Tokio Contributors. *Tokio: an asynchronous runtime for the Rust programming language*. URL: <https://tokio.rs/> (дата обр. 20.04.2025).
- [19] University of California, Berkeley. *BOINC: open-source software for volunteer computing*. URL: <https://boinc.berkeley.edu/> (дата обр. 20.04.2025).
- [20] В. В. Топорков и Д. М. Емельянов. “Экономическая модель планирования и справедливого разделения ресурсов в распределённых вычислениях”. В: *Программирование* 40.1 (2014), с. 35—42. DOI: [10.1134/S0361768814010071](https://doi.org/10.1134/S0361768814010071).